UNIT-I

Introduction to DBMS and Basics of SQL

Topics:

INTRODUCTION: Database-System Applications, Characteristics, Purpose of Database Systems, View of Data, Database Languages, Data Storage and Querying, Transaction Management, Data Base Architecture, Database Users and Administrators.

Basic SQL: Simple Database Schema, data types, table definitions (create, alter), different DML operations (insert, delete, update)

CHAPTER-I INTRODUCTION

DBMS DEFINITION

A database-management system (DBMS) is a collection of interrelated data and a set of programs to access those data. The collection of data, usually referred to as the database, contains information relevant to an enterprise.

The primary goal of a DBMS is to provide a way to store and retrieve database information that is both convenient and efficient.

1.1 Database-System Applications

Database systems are designed to support a wide range of real-world applications where large volumes of data must be stored, updated, and accessed efficiently by multiple users. These applications rely on databases to ensure data accuracy, consistency, security, and availability.

Here are some representative applications:

1. **Enterprise Information Systems**
   In enterprises, databases are used to manage core business functions. Sales databases store customer details, product information, and purchase records. Accounting systems maintain payments, receipts, assets, liabilities, and account balances. Human resource databases store employee records, salary details, payroll information, taxes, and benefits.
2. **Manufacturing Applications**
   Manufacturing industries use databases to manage supply chains, track production processes, monitor inventories in warehouses and stores, and process customer orders. These databases help organizations improve efficiency and reduce costs.
3. **Banking and Financial Systems**
   Banks depend heavily on databases to store customer information, account details, loans, and transaction records. Credit card systems use databases to record purchases and generate monthly

statements. Financial institutions also store data about stocks, bonds, and other financial instruments, including real-time market data for online and automated trading.

4. **Educational Institutions**
   Universities and colleges use database systems to manage student information, course registrations, grades, and examination records. Databases also support administrative functions such as human resources and accounting within educational institutions.
5. **Airline Reservation Systems**
   Airlines were among the earliest users of distributed database systems. Databases store flight schedules, seat availability, passenger reservations, and ticketing information, enabling real-time access across multiple locations.
6. **Telecommunication Systems**
   Telecommunication companies use databases to record call details, text messages, and data usage. These databases support billing systems, prepaid balance management, and maintenance of communication network information.
7. **Web-Based Applications**
   Modern web services rely extensively on databases. Social media platforms store user profiles, connections, posts, and likes. Online retailers use databases to manage sales and orders, as well as to track user behavior for recommendations. Online advertising systems store click histories to enable targeted advertisements.
8. **Document and Navigation Databases**
   Document databases manage large collections of news articles, research papers, and patents. Navigation systems use databases to store locations of places, road networks, and transportation routes, supporting accurate and real-time navigation services.

1.2 Characteristics

A database system is software designed to store, manage, and retrieve large amounts of data efficiently. Unlike simple file systems, database systems support concurrent access by multiple users, ensure data security and consistency, and handle complex data structures.

i. **Highly Valuable:** Database data represents critical organizational information. Its accuracy, security, and availability directly affect business operations and decision-making.
ii. **Large in Volume:** Databases store massive and continuously growing amounts of data. Efficient storage, indexing, and retrieval techniques are required to handle this scale.
iii. **Accessed Concurrently:** Multiple users and applications access the same data simultaneously. Database systems ensure consistency and correctness through concurrency control.
iv. **Reliable and Secure:** Data must remain consistent even during failures and be protected from unauthorized access.
v. **Always Available:** Database systems are designed for high availability and fast response to support continuous operations.

1.3. Purpose of Database Systems

Traditional file-processing systems store organizational data in separate files managed by individual application programs. As data volume and application needs grow, such systems become complex and inefficient, leading to problems like data redundancy, inconsistency, poor data access, and security issues. Database systems were developed to overcome these limitations by providing centralized, reliable, and secure data management.

University File-Processing Example

In a traditional file-processing system, a university stores data about students, instructors, departments, and courses in operating-system files. Separate application programs are written to add students and courses, register students, assign grades, calculate GPA, and generate transcripts.

As new requirements arise (such as introducing a new major), new files and programs must be created, increasing system complexity. This typical file-processing system is supported by a conventional operating system.

Disadvantages of File-Processing Systems(File System Vs DBMS)

i. **Data Redundancy and Inconsistency**
   In file-processing systems, the same data is often stored in multiple files created by different programmers over time. This duplication increases storage cost and may lead to inconsistencies when updates are not applied uniformly across all files.

ii. **Difficulty in Accessing Data**
   File-based systems require a separate application program for each type of data request. If a new query is needed, a new program must be written, making data retrieval slow, inflexible, and inefficient.

iii. **Data Isolation**
   Data is scattered across many files with different formats. This makes it difficult to combine data from multiple files or write new programs to retrieve required information.

iv. **Integrity Problems**
   Consistency constraints (such as non-negative balances or course limits) are enforced through application code. Adding or modifying constraints is difficult, especially when multiple files are involved.

v. **Atomicity Problems**
   In case of system failures, file-processing systems cannot easily ensure that operations are completed fully or not at all. Partial updates can leave the data in an inconsistent state.

vi. **Concurrent-Access Anomalies**
   When multiple users access and update data simultaneously, lack of proper coordination may result in incorrect or inconsistent data values.

vii. **Security Problems**

File-processing systems provide limited control over user access. Enforcing different access privileges for different users is difficult and error-prone.

Traditional File System VS DBMS

Feature	File System	DBMS
Data Storage	Separate files	Centralized database
Redundancy	High	Low
Consistency	Not guaranteed	Maintained
Data Access	Program dependent	Flexible queries
Data Isolation	Data scattered	Integrated data
Integrity	Application-level	System-enforced
Atomicity	Not ensured	Guaranteed
Concurrency	Poor control	Proper control
Security	Weak	Strong
Maintenance	Difficult	Easier

1.4 View of Data

A database system consists of a collection of interrelated data and a set of programs that enable users to access and modify this data. Its primary purpose is to provide an abstract view of data by hiding details of data storage and maintenance from users.

Data Models

A data model defines the structure of a database. It provides conceptual tools to describe data, relationships among data, data meaning, and consistency constraints.

Types of Data Models

- **Relational Data Model**

  The relational model represents data using tables (relations). Each table consists of rows (records) and columns (attributes), with each column having a unique name. It is a record-based model with fixed-format records and is the most widely used data model in modern database systems.

- **Entity–Relationship (E-R) Model**
  The E-R model represents data using entities (real-world objects) and relationships among them. It is mainly used for database design to model real-world scenarios clearly before implementing the database.
- **Semi-Structured Data Model**
  This model supports data with flexible and varying structures, where similar data items may have different attributes. JSON and XML are common examples. It is suitable for web and document-based applications.
- **Object-Based Data Model**
  This model integrates object-oriented concepts such as encapsulation, methods, and object identity into databases. Objects and procedures can be stored and executed within the database, extending the relational model to support modern programming paradigms.

Data Abstraction

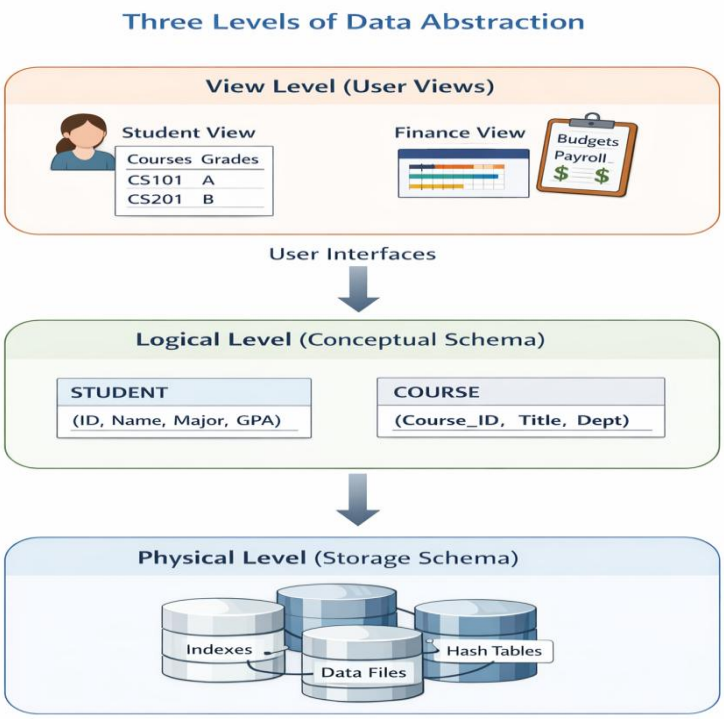
To retrieve data efficiently, database systems use complex data structures. Since most users are not technically trained, this complexity is hidden using multiple levels of data abstraction, simplifying user interaction with the system.

There are three levels of Data Abstraction.

i. **Physical Level**
   This is the lowest level of abstraction. It describes how data is actually stored on storage devices using complex, low-level data structures and file organizations.

ii. **Logical Level**
   The logical level describes what data is stored in the database and the relationships among the data. It presents the entire database using simple structures while hiding physical storage details. This separation provides *physical data independence*. Database administrators primarily work at this level.

iii. **View Level**
   The view level is the highest level of abstraction. It shows only a selected portion of the database to users, hiding unnecessary details. Multiple views can exist for the same database, each tailored to the needs of different users or applications.



Database Schema and Instance (Important Terminology)

Database Instance

A database instance is the collection of data stored in the database at a specific moment in time. It changes frequently as data is inserted, updated, or deleted.

Database Schema

The database schema represents the overall design or structure of the database. It defines how data is organized and remains relatively stable over time.

Schema–Instance Analogy

A database schema is similar to variable declarations and data types in a program, while a database instance is like the actual values of those variables at a given time.

Types of Schemas

Database systems maintain multiple schemas based on levels of abstraction:

- **Physical Schema:** Describes how data is stored at the physical level.
- **Logical Schema:** Describes what data is stored and the relationships among data.
- **View Schemas (Subschemas):** Define different user views of the database.

Importance of Logical Schema

The logical schema is most important for application development, as programmers use it to construct database applications.

Physical Data Independence

Since the physical schema is hidden from applications, changes at the physical level do not require modifications to application programs. This property is known as ***physical data independence***.

1.5. Database Languages

A database system provides two main language components: a **Data-Definition Language (DDL)** to define the database schema and a **Data-Manipulation Language (DML)** to query and update data. In practice, both are part of a single database language, most commonly **SQL**, which is used by almost all relational database systems.

i. Data-Definition Language (DDL)

DDL is used to define the structure of the database, including tables, attributes, data types, and storage details. It also specifies **integrity constraints** that ensure data consistency.

Types of Integrity Constraints

- **Domain Constraints:** Restrict attribute values to a specified domain (data type, range, format).
- **Referential Integrity:** Ensures relationships between tables remain valid (e.g., a department referenced in a course must exist).
- **Authorization:** Controls user access (read, insert, update, delete privileges).

Data Dictionary

The execution of DDL statements stores metadata in the **data dictionary**, which contains information about database structure and constraints. It is maintained internally by the DBMS.

ii. SQL Data-Definition Language

SQL allows the creation of tables with defined data types and constraints. It supports primary keys, foreign keys, and authorization rules to maintain data integrity and security.

iii. Data-Manipulation Language (DML)

DML allows users to retrieve and modify data in the database. Its operations include data retrieval, insertion, deletion, and modification.

Types of DML

- **Procedural DML:** Specifies *what* data is needed and *how* to obtain it.
- **Declarative (Nonprocedural) DML:** Specifies *what* data is needed without describing *how* to retrieve it (e.g., SQL).

Queries

" A query is a request for data retrieval. SQL queries operate on one or more tables and return results in tabular form" .

iv. SQL Data-Manipulation Language

SQL is a nonprocedural query language. It allows users to specify conditions and attributes to retrieve data from single or multiple tables without specifying access methods.

v. Database Access from Application Programs

SQL alone cannot perform all computations or handle user interaction. Therefore, SQL is embedded in host programming languages such as C, Java, or Python. These **application programs** interact with the database to perform tasks like registration, payroll processing, and report generation.

Database Interfaces

- **ODBC:** Standard interface for database access using C and other languages.
- **JDBC:** Java-based interface for database connectivity.

1.6. Data Storage and Querying

A **database system** is divided into modules to handle its core responsibilities: **Storage Manager**, **Query Processor**, and **Transaction Manager**. These components enable efficient storage, retrieval, and management of large-scale data.

i. Storage Manager

The storage manager interfaces between application programs/queries and the low-level data stored on disks. It manages:

- **Authorization & Integrity Manager:** Enforces integrity constraints and user access rights.
- **Transaction Manager:** Ensures database consistency during system failures and concurrent transactions.
- **File Manager:** Handles disk space allocation and file structures.
- **Buffer Manager:** Fetches data from disk to main memory and manages caching for efficiency.

Key Data Structures:

- **Data Files:** Store actual database data.
- **Data Dictionary:** Stores metadata about database schema and structure.
- **Indices:** Provide fast access to data items based on attribute values.

Databases often store data from hundreds of gigabytes to petabytes. Efficient disk-memory management is crucial, with SSDs increasingly used for speed.

ii. Query Processor

The query processor simplifies data access and ensures efficient execution of queries and updates. Its components include:

- **DDL Interpreter:** Processes DDL statements and updates the data dictionary.
- **DML Compiler:** Translates DML statements into low-level instructions and generates evaluation plans.
- **Query Evaluation Engine:** Executes instructions generated by the DML compiler.

Query Optimization: The DML compiler selects the most efficient evaluation plan from multiple alternatives, ensuring optimal performance.

1.7. Transaction Management

Transaction Definition:

"A **transaction** is a collection of operations that performs a single logical task in a database. It is treated as a unit of **atomicity** and **consistency**."

Key Properties of Transactions (ACID):

i. **Atomicity:** A transaction must execute fully or not at all. Partial execution is not allowed.
ii. **Consistency:** A transaction must preserve database consistency; if the database is consistent before a transaction, it must remain consistent after successful completion.
iii. **Durability:** Once a transaction completes successfully, its effects persist, even in the event of system failure.

Example:

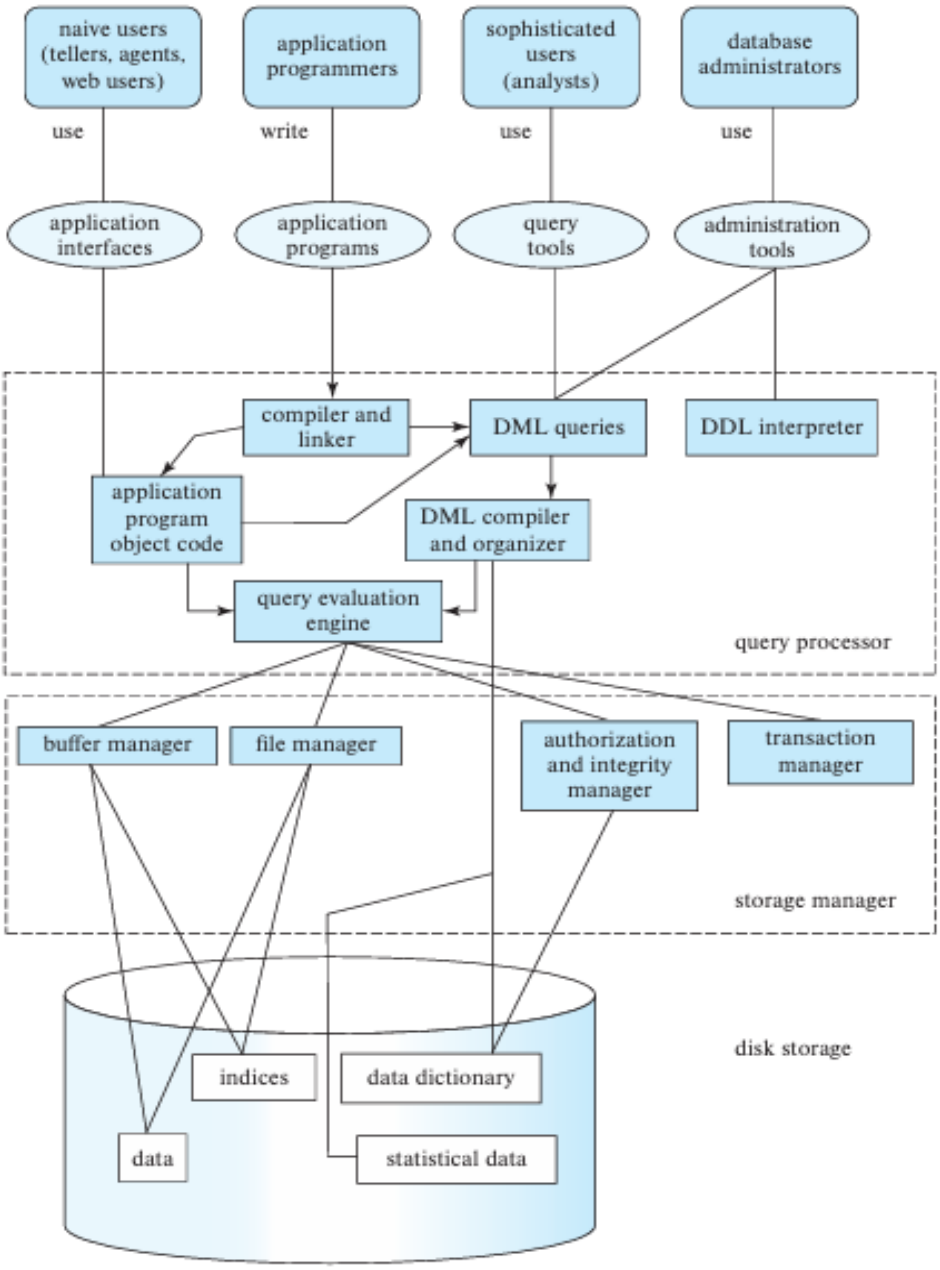
A funds transfer from account A to account B involves two operations: debit A and credit B. Both must succeed together (atomicity), preserve total balances (consistency), and survive system crashes (durability).

Temporary Inconsistency: During execution, a transaction may temporarily leave the database in an inconsistent state (e.g., debit occurs before credit). Proper transaction design ensures consistency is restored upon completion.

Responsibilities in Transaction Management:

- **Programmer:** Defines transactions correctly to preserve consistency.
- **Database System:** Ensures atomicity and durability through the **recovery manager**, which performs failure recovery if needed.
- **Concurrency-Control Manager:** Manages interactions among concurrent transactions to maintain consistency.

1.8. Database Architecture



A **Database System Architecture** describes how different users interact with the database system and how requests are processed internally to access stored data efficiently, securely, and reliably.

The architecture consists of **users**, **interfaces**, **query processor**, **storage manager**, and **disk storage**.

Database Users and Their Interfaces

Types of Users

i. **Naïve Users**
   a. Examples: Tellers, agents, web users

b. Do not write queries
c. Interact using predefined forms and applications

ii. **Application Programmers**
a. Write database applications using programming languages (Java, Python, C, etc.)
b. Embed SQL queries inside programs

iii. **Sophisticated Users (Analysts)**
a. Directly write and execute SQL queries
b. Use query tools for data analysis

iv. **Database Administrators (DBA)**
a. Manage database structure, security, backup, and recovery
b. Use administrative tools

Query Processor

The **Query Processor** translates high-level queries into low-level instructions that the storage manager can execute.

Components of Query Processor

a) DDL Interpreter

- Processes **DDL (Data Definition Language)** commands
- Examples: CREATE, ALTER, DROP
- Stores schema definitions in the **Data Dictionary**

b) DML Compiler and Organizer

- Converts **DML (Data Manipulation Language)** queries into execution plans
- Examples: SELECT, INSERT, UPDATE, DELETE
- Optimizes queries for efficient execution

c) Query Evaluation Engine

- Executes the optimized query plan
- Interacts with the storage manager to fetch or modify data

d) Compiler and Linker

- Converts application programs with embedded SQL into object code
- Links database calls with runtime support

Storage Manager

The **Storage Manager** acts as an interface between the query processor and disk storage.

Components of Storage Manager

a)Buffer Manager

- Manages data transfer between disk and main memory
- Improves performance by caching frequently used data

b) File Manager

- Manages storage space on disk
- Handles file organization and access methods

c) Authorization and Integrity Manager

- Enforces security and access control
- Maintains integrity constraints (primary key, foreign key, etc.)

d) Transaction Manager

- Ensures **ACID properties**
  - Atomicity
  - Consistency
  - Isolation
  - Durability
- Handles concurrency control and recovery

Disk Storage Components

The database is physically stored on disk and consists of the following:

i. Data

- Actual stored records (tables)

ii. Indices

- Data structures (B-tree, hash index)
- Improve query performance

iii. Data Dictionary

Stores metadata:

- Table schemas
- Attributes
- Constraints
- User privileges

iv. Statistical Data

- Information about data distribution
- Used by the query optimizer

Flow of Query Processing (Step-by-Step)

1. User submits a request through an interface
2. Query reaches the **Query Processor**
3. DDL queries → DDL Interpreter
   DML queries → DML Compiler and Organizer
4. Optimized query sent to **Query Evaluation Engine**
5. Storage Manager retrieves required data from disk
6. Results are returned to the user

Database Application Architectures

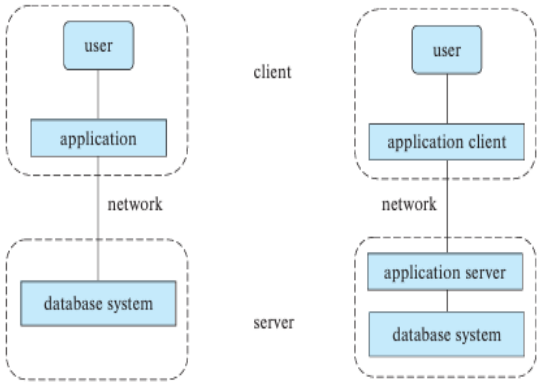
Database applications can be organized into **two-tier** or **three-tier architectures**, depending on how the client, application, and database interact.

1. Two-Tier Architecture

- **Structure:**
  - Client machine to Database server
- **Characteristics:**
  - The application resides on the **client machine**.
  - Client sends **query language statements** (e.g., SQL) directly to the database server.
  - Common in **earlier-generation database applications**.
- **Limitations:**
  - Business logic distributed across multiple clients.
  - Security and performance are relatively lower.

2. Three-Tier Architecture

- **Structure:**
  - **Client (Front-end)** – Web browsers, mobile apps
  - **Application Server (Middle tier)** – Contains business logic
  - **Database Server (Back-end)** – Stores and manages data
- **Characteristics:**
  - Client acts only as a **frontend**, does **not make direct database calls**.
  - **Application server** mediates between client and database.
  - Business logic is centralized in the **application server**.
- **Advantages:**
  - **Better security** – direct database access is restricted.
  - **Better performance** – centralized processing reduces redundancy.
  - Easier to **maintain and scale**.



(a) Two-tier architecture (b) Three-tier architecture

1.9 Database Users and Administrators

1. Database Users and Interfaces

Types of Users:

i. Naïve Users:
   a. Unsophisticated users using predefined interfaces (web/mobile apps).
   b. Interact via forms or reports.
   c. Example: Students registering for classes via a web interface that connects to the database.

ii. Application Programmers:
   a. Develop applications and user interfaces that interact with the database.
   b. Use programming tools and APIs to create efficient user interfaces.

iii. Sophisticated Users:
   a. Interact with the database directly without programming.
   b. Use query languages (e.g., SQL) or data analysis tools.
   c. Examples: Analysts exploring trends or generating reports.

2. Database Administrator (DBA)

Role: Central control over data and programs accessing the database.

Key Functions:

i. Schema Definition:
   a. Create the original database schema using DDL statements.

ii. Storage & Access Method Definition:
   a. Specify physical storage organization and indices.

iii. Schema & Physical Modifications:
   a. Update schema or storage for changing requirements or performance improvements.

iv. Authorization Management:
   a. Grant and control user access to data.
   b. Maintain authorization records in the system structure.

v. Routine Maintenance:
   a. Backup database to prevent data loss.
   b. Monitor and manage disk space.
   c. Ensure performance by monitoring jobs and controlling resource-heavy tasks.

CHAPTER-II Basic SQL

1.10 Simple Database Schema

A **database schema** represents the **overall logical design** of a database. It describes **what data exists**, **how it is related**, and **how it is constrained**.

What a Schema Contains

- Tables
- Columns and data types
- Relationships between tables
- Constraints (PK, FK, UNIQUE, etc.)
- Views, indexes, procedures

Why Schema Is Important

- Acts as a **blueprint** before data storage
- Ensures **consistency and integrity**
- Helps in **database security** (user access control)
- Makes databases **scalable and maintainable**

Real-World Analogy

"A schema is like the **architectural plan of a building** before construction."

Example

```
CREATE SCHEMA university;
```

Table inside schema:

```
CREATE TABLE university.students (student_id INT,name VARCHAR(50));
```

1.10.SQL Datatypes

Data types define the **nature, size, and range of data** that can be stored in a column.

i. Numeric Data Types

Data Type	Usage Example
INT	Roll numbers, IDs
SMALLINT	Age
BIGINT	Bank account numbers
FLOAT	Scientific values
DECIMAL(10,2)	Salary

Example : **`salary DECIMAL(10,2)`**

ii. Character / String Data Types

Data Type	Description
CHAR(n)	Fixed size
VARCHAR(n)	Variable size
TEXT	Long descriptions

Example: `name VARCHAR(50),gender CHAR(1)`

iii. Date and Time Data Types

Data Type	Purpose
DATE	Birth date
TIME	Login time
DATETIME	Event date & time
TIMESTAMP	Automatic system time

Example: `admission_date DATE`

iv. Boolean Data Type

Data Type	Values
BOOLEAN	TRUE / FALSE

Example: `is_active BOOLEAN`

1.11. Table Definition

A **table** is the **basic unit of data storage** in a relational database with rows and columns.

Here..

- Rows are Records
- Columns are Attributes

i. Creating a Table with Constraints

Syntax:

```
CREATE TABLE table_name (
    column1 datatype [constraint],
    column2 datatype [constraint],
    ...
    [table_constraint]
);
```

Example:

```
CREATE TABLE students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    age INT CHECK (age >=18),
    email VARCHAR(100) UNIQUE,
    department VARCHAR(30) DEFAULT'CSE',
    admission_date DATE
);
```

Constraints

Constraint	Meaning
PRIMARY KEY	Unique + Not Null
FOREIGN KEY	Reference another table
UNIQUE	Prevent duplicates
NOT NULL	Mandatory field
CHECK	Validation rule
DEFAULT	Default value

Foreign Key Example

```
CREATE TABLE enrollments (
    enroll_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES students(student_id)
);
```

ii. Alter table:

ALTER commands are part of DDL (Data Definition Language) and are used to modify existing database tables.

Syntax:

Operation	Syntax
Add a column	`ALTER TABLE table_name ADD column_name datatype [constraint];`

Add multiple columns	ALTER TABLE table_name ADD (column1 datatype, column2 data type, ...);
Modify datatype / size	ALTER TABLE table_name MODIFY column_name new_datatype;
Rename a column	ALTER TABLE table_name RENAME COLUMN old_name TO new_name;
Drop a column	ALTER TABLE table_name DROP column_name;
Add PRIMARY KEY	ALTER TABLE table_name ADD CONSTRAINT constraint_name PRIMARY KEY (column_name);
Add FOREIGN KEY	ALTER TABLE table_name ADD CONSTRAINT constraint_name FOREIGN KEY (column_name) REFERENCES parent_table(column);
Add UNIQUE constraint	ALTER TABLE table_name ADD CONSTRAINT constraint_name UNIQUE (column_name);
Add CHECK constraint	ALTER TABLE table_name ADD CONSTRAINT constraint_name CHECK (condition);
Drop constraint	ALTER TABLE table_name DROP CONSTRAINT constraint_name;
Disable constraint	ALTER TABLE table_name DISABLE CONSTRAINT constraint_name;
Rename table	ALTER TABLE table_name RENAME TO new_table_name;

Examples:

Operation	SQL Example
Add column	ALTER TABLE students ADD email VARCHAR(100);
Drop column	ALTER TABLE students DROP COLUMN department;
Rename column	ALTER TABLE students RENAME COLUMN name TO full_name;
Rename table	ALTER TABLE students RENAME TO learners;
Modify data type	ALTER TABLE students ALTER COLUMN age TYPE SMALLINT;
Set NOT NULL	ALTER TABLE students ALTER COLUMN email SET NOT NULL;
Drop NOT NULL	ALTER TABLE students ALTER COLUMN email DROP NOT NULL;

Set default	`ALTER TABLE students ALTER COLUMN department SET DEFAULT 'CSE';`
Drop default	`ALTER TABLE students ALTER COLUMN department DROP DEFAULT;`
Add constraint	`ALTER TABLE students ADD CONSTRAINT unique_email UNIQUE(email);`
Drop constraint	`ALTER TABLE students DROP CONSTRAINT unique_email;`

1.12 DML Commands (Data Manipulation Language)

DML commands **work on the data**, not the structure.

i. INSERT – Adding Records

Insert full row:

Syntax:

```
INSERT INTO table_name VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO students VALUES (1, 'Anita', 20, 'anita@gmail.com', 'ECE', '2024-06-10');
```

Insert selected columns:

Syntax:

```
INSERT INTO table_name (column1, column2, ...) VALUES (value1, value2, ...);
```

Example:

```
INSERT INTO students (student_id, name, age) VALUES (2, 'Rahul', 19);
```

ii. SELECT – Retrieving Records

Fetch all data:

Syntax:

```
SELECT * FROM table_name;
```

Example:

```
SELECT * FROM students;
```

Fetch selected columns:

Syntax:

```
SELECT column1, column2, ... FROM table_name WHERE condition;
```

Example:

```
SELECT name, department FROM students;
```

Using WHERE:

Syntax:

```
SELECT column1, column2, ... FROM table_name WHERE condition;
```

Example:

```
SELECT * FROM students WHERE age >19;
```

iii. UPDATE – Modifying Records

Syntax:

```
UPDATE table_name SET column1 = value1, column2 = value2 WHERE condition;
```

Examples:

```
UPDATE students SET department ='MECH'WHERE student_id =2;
```

Update multiple columns:

```
UPDATE students SET age =21, department ='CSE'WHERE student_id =1;
```

iv. DELETE – Removing Records

Delete specific row:

Syntax:

```
DELETE FROM table_name WHERE condition;
```

Example:

```
DELETE FROM students WHERE student_id =2;
```

Delete all rows:

Syntax:

```
DELETE FROM table_name;
```

Example:

```
DELETE FROM students;
```

Above DML Commands in Table View:

Command	Syntax	Example
INSERT	Full row	`INSERT INTO students VALUES (...);`
INSERT	Selected columns	`INSERT INTO students (student_id, name, age) VALUES (...);`
SELECT	All data	`SELECT * FROM students;`
SELECT	Specific columns	`SELECT name, department FROM students;`
SELECT	With condition	`SELECT * FROM students WHERE age>19;`
UPDATE	Single column	`UPDATE students SET department='MECH' WHERE student_id =2;`
UPDATE	Multiple columns	`UPDATE students SET age=21, department='CSE' WHERE student_id=1;`
DELETE	Specific row	`DELETE FROM students WHERE student_id=2;`
DELETE	All rows	`DELETE FROM students;`

DML vs DDL

Feature	DDL	DML
Purpose	Structure	Data
Commands	CREATE, ALTER, DROP	INSERT, SELECT, UPDATE, DELETE
Auto Commit	Yes	No

Prepared By
Dr. C.V. Lakshmi Narayana
Assistant Professor
Department of AI&ML
Annamacharya University, Rajampet